



Reto Técnico Back End

API Gateway + Microservicios + OAuth 2.0





Funcionalidades requeridas

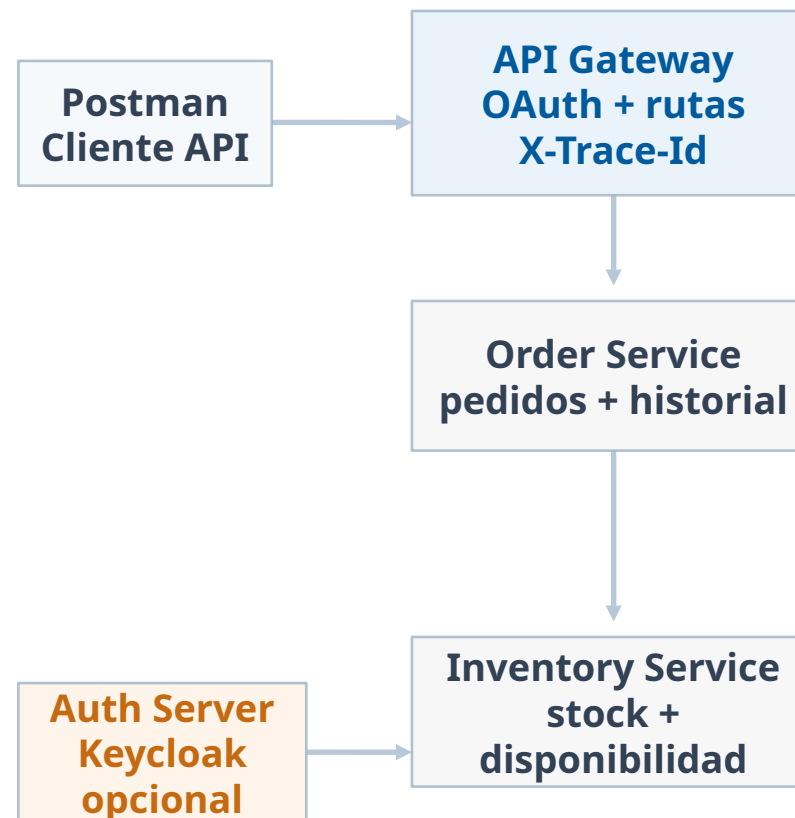
- Crear una API backend para registrar y consultar pedidos de productos.
- Permitir registrar un pedido, consultar su estado, consultar su historial y cancelar un pedido cuando aplique.
- El uso de la API debe poder probarse completamente desde Postman.
- Para acceder a la API debe existir un mecanismo de autenticación.
- No se debe desarrollar frontend ni BFF; el único punto público será el API Gateway.
- El API Gateway debe validar seguridad, enrutar solicitudes y propagar el identificador de tracking.
- Crear un microservicio de pedidos y un microservicio de inventario.
- Debe existir un identificador para tracking desde el API Gateway hasta cada microservicio.





Arquitectura requerida

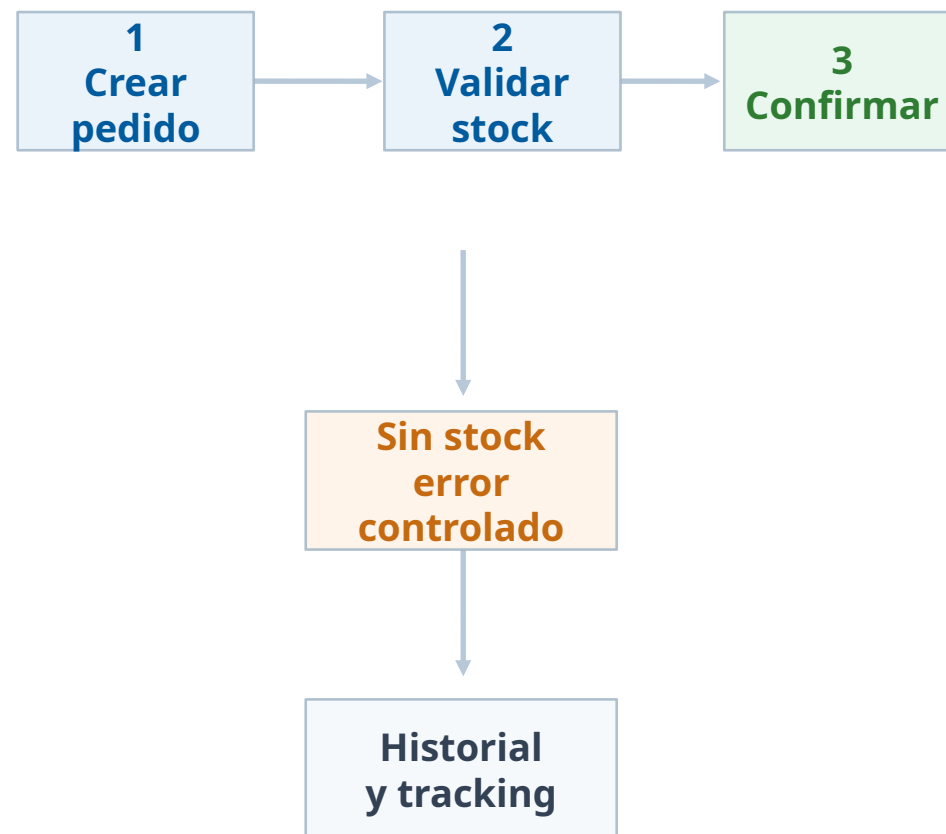
- La arquitectura debe estar compuesta por API Gateway, Order Service e Inventory Service.
- El API Gateway será el único punto público del sistema.
- El API Gateway no debe contener reglas de negocio ni componer respuestas como un BFF.
- Order Service debe administrar pedidos, estados e historial.
- Inventory Service debe administrar productos, stock y disponibilidad.
- Cada microservicio debe ser responsable de su propia información y persistencia.
- La comunicación entre servicios podrá ser síncrona mediante HTTP reactivo usando WebClient.





Flujo simplificado de pedido

- El flujo principal inicia con la solicitud de creación del pedido.
- Order Service valida datos y consulta disponibilidad en Inventory Service.
- Si existe stock, se registra el pedido y queda en estado CONFIRMED.
- Si no existe stock, se responde STOCK_INSUFFICIENT y el pedido no se confirma.
- La cancelación solo debe permitirse para estados definidos por la solución.
- Order Service debe registrar el historial de cambios de estado.
- No se exige Saga Orchestrator, compensaciones distribuidas ni Dispatch Service.





Estados, endpoints y errores

- Estados de pedido mínimos: PENDING, CONFIRMED y CANCELLED; FAILED puede manejarse como opcional.
- No se deben permitir transiciones inválidas ni cancelaciones duplicadas.
- La API debe permitir registrar, consultar y cancelar pedidos, además de consultar su historial.
- El candidato definirá rutas, métodos HTTP, códigos y contratos, y documentará la API con OpenAPI/Swagger y Postman. Order Service consultará la disponibilidad en Inventory Service.
- Errores mínimos: pedido inexistente, datos inválidos, stock insuficiente, transición inválida, token inválido o expirado y error interno.
- El formato de error debe ser: { timestamp, status, code, message, traceId }.
- El API Gateway debe generar o recibir X-Trace-Id, propagarlo e incluirlo en logs, respuestas y errores.





Consideraciones técnicas

- Usar patrones de diseño y principios SOLID.
- La información debe estar almacenada en la base de datos de su preferencia.
- Implementar los microservicios en Java 17 o superior con Spring Boot, Spring Security, Spring Data, JUnit 5, AOP y WebFlux.
- Implementar el API Gateway con Spring Cloud Gateway o tecnología equivalente.
- La seguridad debe implementarse con OAuth 2.0 u OpenID Connect con JWT; se permite Keycloak en Docker.
- Dockerizar API Gateway, microservicios, bases de datos y servidor de autenticación.
- Configurar Logback e incluir X-Trace-Id en los logs.
- Persistir pedidos, historial, inventario y estados de pedido.
- No es obligatorio implementar Saga, Kafka, Outbox, Event Sourcing, CQRS, Kubernetes, compensaciones ni Dispatch Service.
- Usar MapStruct, Lombok, Gson o Jackson; documentar README.md y diseñar API con OpenAPI / Swagger.
- La solución debe ejecutarse con docker compose up --build o comando equivalente.

